

Introduction to Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with text generation to produce more accurate and grounded responses. RAG systems first retrieve relevant documents from a knowledge base, then use a language model to generate answers based on the retrieved context.

Key Components of RAG:

1. Document Ingestion: Documents are loaded, split into chunks, and indexed in a vector store for efficient similarity search.
2. Embedding Model: Text chunks are converted into dense vector representations using models like sentence-transformers/all-MiniLM-L6-v2.
3. Vector Store: ChromaDB or similar databases store embeddings and allow fast nearest neighbor retrieval based on cosine similarity.
4. Language Model: An LLM such as Llama 3.2 generates natural language answers using the retrieved context as grounding information.

Benefits of RAG over pure LLM approaches:

- Reduced hallucination by grounding responses in actual documents
- Ability to update knowledge without retraining the model
- Domain-specific expertise from custom document collections
- Transparent sourcing with traceable references to original documents

Architecture and Implementation Details

The RAG pipeline follows a structured workflow:

Step 1 - Document Processing:

PDF documents are parsed using libraries such as PyPDF. The extracted text is then split into manageable chunks using RecursiveCharacterTextSplitter with configurable chunk_size and chunk_overlap parameters. Typical values are 500 characters per chunk with 50 characters of overlap to maintain context continuity.

Step 2 - Indexing:

Each text chunk is embedded using a sentence transformer model and stored in ChromaDB. The vector store persists to disk so that re-indexing is not required on every startup.

Step 3 - Retrieval:

When a user submits a query, the system embeds the query using the same model and performs a similarity search in the vector store. The top-k most relevant chunks are returned as context for the generation step.

Step 4 - Generation:

The retrieved chunks are formatted into a prompt template along with the user question. The prompt is sent to an Ollama-hosted LLM which generates a contextual answer.

Deployment:

The system is containerized using Docker Compose with separate services for the Ollama model server and the Flask web application. A shared volume persists the ChromaDB data. The Flask API exposes a POST /ask endpoint and serves a chat UI at the root path.